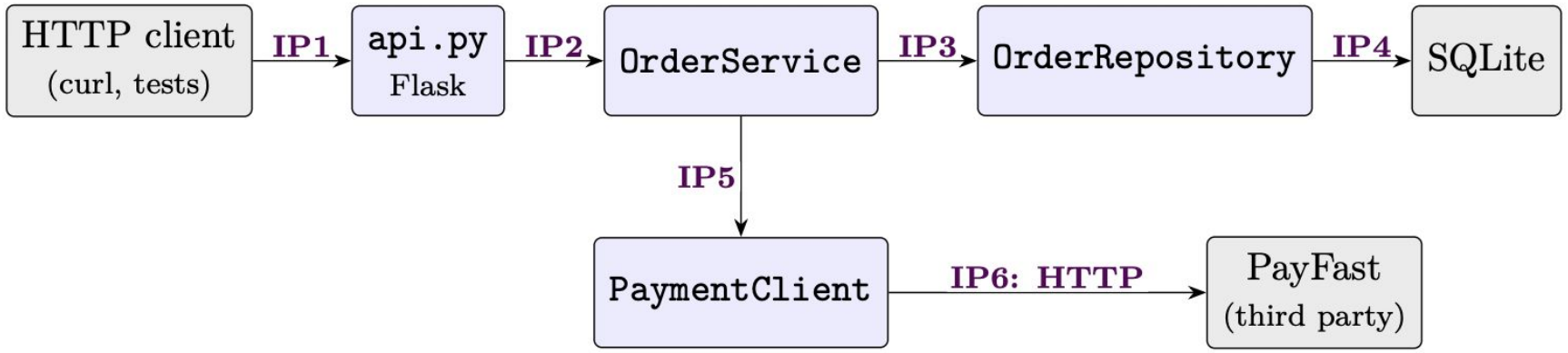
Integration Points in the Shop Demo: One Seam, One Test

Representative tests for each interaction point, side by side with the code they exercise

SE 413/513 Software Testing · Demo: demos/integration_testing

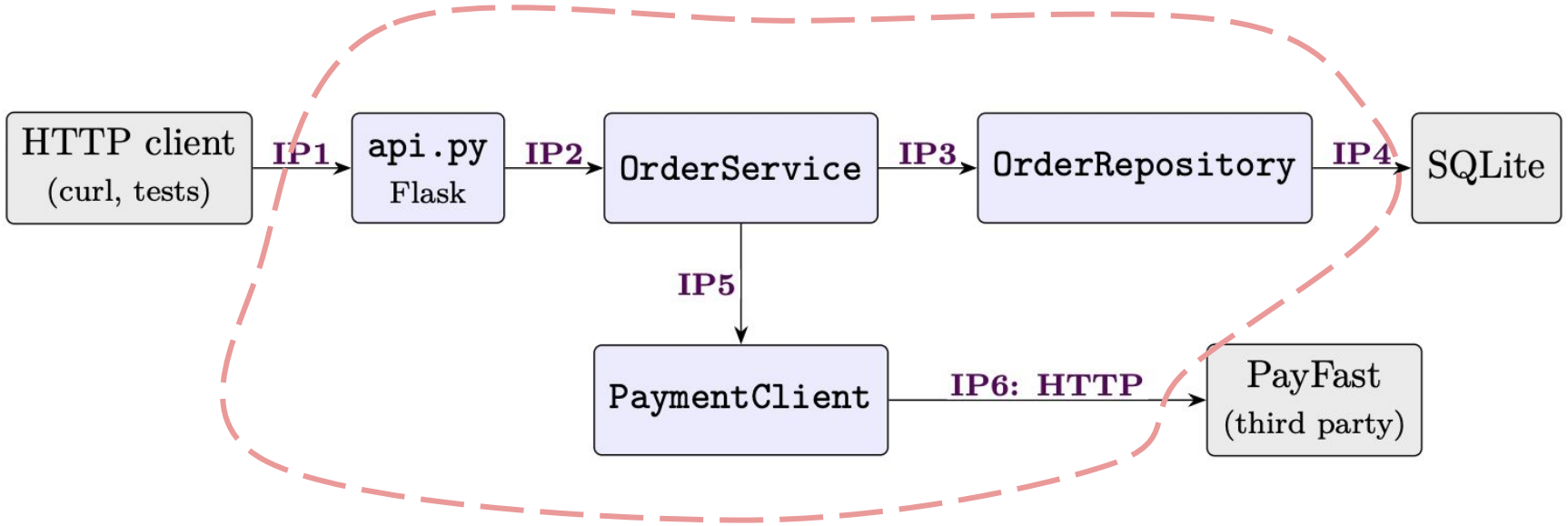
Dr. Kamrul Hasan, Grand Valley State University

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

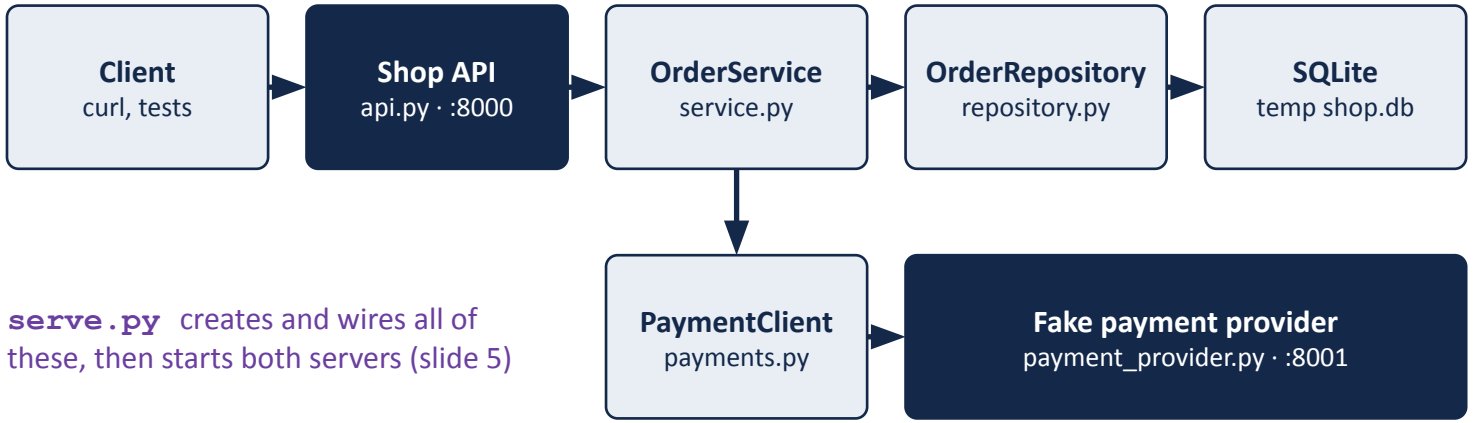
Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

How the pieces fit

Source
`shop/` · `serve.py` · `fakes/`



`serve.py` creates and wires all of these, then starts both servers (slide 5)

API `api.py`
HTTP concerns only: API key, validation, routes, JSON, status codes.

Service `service.py`
Business operations: reserve stock, charge, undo if the charge fails.

Repository `repository.py`
All the SQL. Never commits; the service decides transactions.

Runner `serve.py`
Local wiring: temp database, fake provider, API key shop-key.

The demo's folder
`demo-testing-suite/demos/integration_testing/`

```
├── serve.py                    runs the shop locally
├── shop/                       the application
│   ├── api.py                  Flask HTTP API
│   ├── service.py              OrderService
│   ├── repository.py           OrderRepository: all SQL
│   ├── db.py                   connect + schema
│   ├── payments.py             PaymentClient
│   ├── errors.py               shared exceptions
│   ├── payments_v1.py          broken: dollars
│   └── service_buggy.py        broken: 2 txns
├── fakes/
│   └── payment_provider.py     fake PayFast
├── specs/                      API contracts + design review
├── tests/                      7 test files + fixtures
└── README.md
```

__init__.py and __pycache__/ not shown

Each layer has one job, which is what makes it testable. create_app(service, api_key) takes the service as a parameter, so a test can hand the API a fake service, or serve.py a per-request one.

Inside api.py: the life of a request

Routes
`POST /orders` · `GET /orders/<id>`

1. **`@app.before_request`**
   require_api_key() compares X-API-Key with the configured key
   → **401** missing or wrong key
2. **`@app.post("/orders")`**
   place_order() reads the body: request.get_json(silent=True)
3. **`validate_order(body)`**
   returns a message if the body breaks a rule, else None
   → **400** message in the body
4. **`service.place_order(...)`**
   sku, quantity, customer_email; business exceptions become statuses
   → **4xx/5xx** see slide 4
5. **`jsonify(order), 201`**
   header Location: /orders/<id>
   → **201** the created order

What validate_order checks

- body is a JSON object (a dict)
- sku, quantity, customer_email are all present
- quantity is an int and at least 1; the exact type check also rejects true/false
- customer_email is a string containing @

Basic checks only: not full email or SKU validation. An unknown SKU is the service's 404.

Bad requests stop as early as possible. The key check runs before any route, validation before the service, so the service only ever sees a well-formed order. GET /orders/<int:id> is simpler: 200 with the order, or 404.

Every response the API can give

Contract
`specs/shop_api.md`

	Status	When	Notes
Success	**201 Created**	POST /orders placed and paid	order + Location header
	200 OK	GET /orders/<id> found it	order JSON
Rejected by api.py	**401 Unauthorized**	X-API-Key missing or wrong	before_request
	400 Bad Request	body breaks a validate_order rule	the rule's message
	404 / 405	unknown route or order id; method not allowed	JSON, never Flask's HTML page
Raised by the service	**404 Not Found**	UnknownProduct: SKU does not exist	
	409 Conflict	OutOfStock: not enough inventory	nothing is charged
	402 Payment Required	PaymentDeclined: card declined	stock is released
	503 Service Unavailable	PaymentUnavailable: provider down	Retry-After: 30
	502 Bad Gateway	PaymentError: provider rejected our request	our side's fault

Every error looks the same

`error(status, message, headers=None)`

```
{"error": "order not found"}
```

Why the except order matters

PaymentDeclined and PaymentUnavailable are subclasses of PaymentError, so place_order catches them first. Listed the other way round, every payment failure would come back 502.

For 402, 502 and 503, the service has already released the reserved stock before the API answers.

A client only ever sees a status code and a JSON body. 502 vs. 503 tells it whose fault a payment failure is: 503 means try again later; 502 means retrying won't help.

Running it locally with serve.py

Stop
Ctrl-C · the database is temporary

1 **Start the fake provider**
create_provider(secret="sk_test_123") on port + 1 (8001), in a daemon thread; --provider-mode sets normal, decline or unavailable.

2 **Create a fresh database**
a temp shop.db, seeded with MUG ($12.50, 5 in stock) and TEE ($20.00, 0 in stock). Prices are integer cents.

3 **One connection per request**
PerRequestService opens a new SQLite connection and OrderService for each call: a SQLite connection belongs to the thread that made it.

4 **Start the shop API**
create_app(PerRequestService(), api_key="shop-key") on 127.0.0.1:8000, until Ctrl-C.

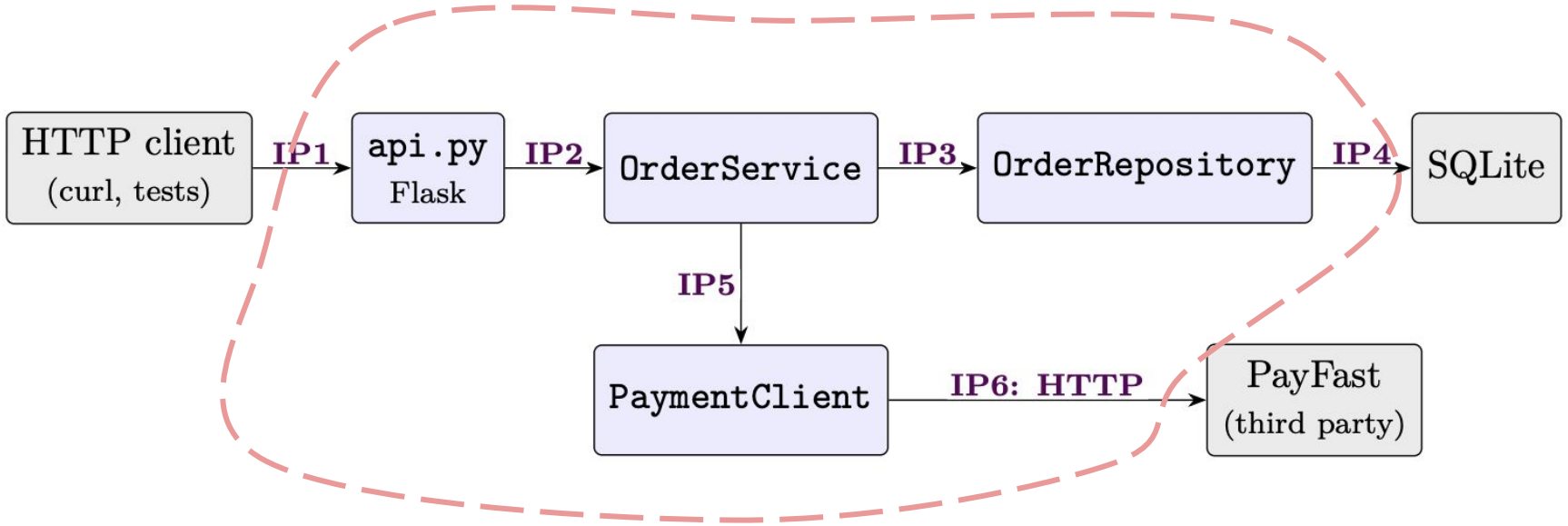
```
$ python serve.py
shop API     http://127.0.0.1:8000   (X-API-Key: shop-key)
fake PayFast http://127.0.0.1:8001   (mode: normal)

$ curl -X POST :8000/orders -H 'X-API-Key: shop-key' \
    -H 'Content-Type: application/json' \
    -d '{"sku":"MUG","quantity":2,
         "customer_email":"ana@example.com"}'
201 CREATED   Location: /orders/1
{"id":1,"sku":"MUG","quantity":2,"total_cents":2500,
 "status":"paid","charge_id":"ch_1", ...}

$ curl :8000/orders/1 -H 'X-API-Key: shop-key'
200 OK   (same order)
no key          → 401 {"error":"missing or invalid API key"}
"sku":"TEE"     → 409 {"error":"insufficient stock"}
"quantity":"2" → 400 quantity must be a positive integer
```

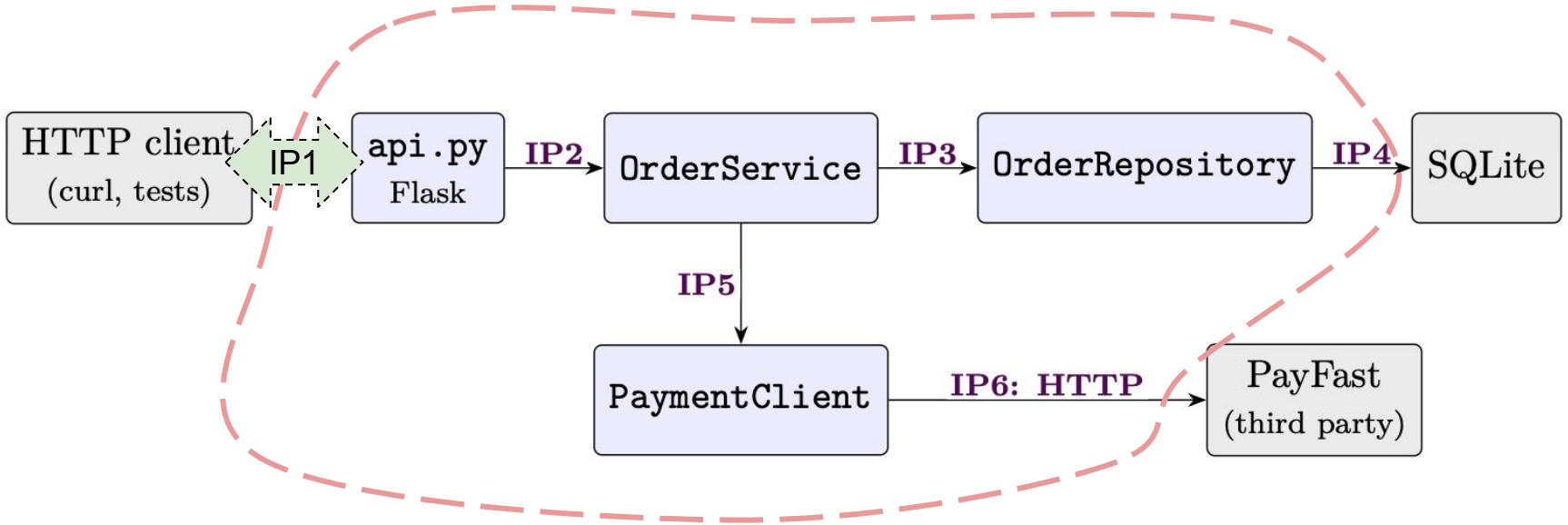
Try a failure: python serve.py --provider-mode decline gives 402; --provider-mode unavailable gives 503 with Retry-After. Each run starts from a fresh, seeded database.

Detailed Integration Testing: Through a Practical Shop Application

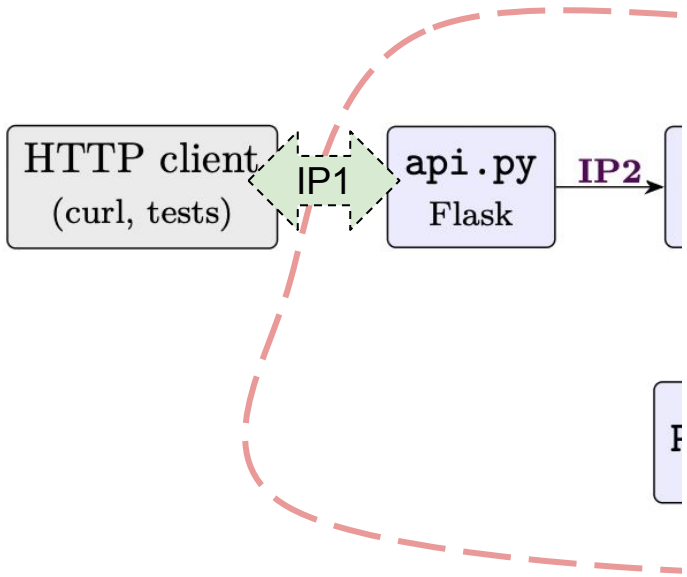


The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.



The shop. Grey boxes are outside the shop's two components must agree.

Shop Orders API, v1

The shop's own HTTP API, implemented in `shop/api.py`. Every request needs the header `X-API-Key: <key>`. Every error response is JSON of the form `{"error": "<message>"}`, never an HTML page.

`POST /orders`

Place an order: reserve stock, charge the customer through PayFast (`payment_provider_api.md`), and record the order.

Body (`Content-Type: application/json`)

Field	Type	Rules
`sku`	string	An existing product.
`quantity`	integer	`>= 1`
`customer_email`	string	Must contain `@`.

Responses

Status	When	Extra
`201 Created`	The order was placed and paid.	Body: the order. Header `Location: /orders/<id>`.
`400 Bad Request`	Body is not a JSON object, a field is missing, or a field breaks a rule.	
`401 Unauthorized`	`X-API-Key` missing or wrong.	
`402 Payment Required`	The provider declined the card.	Stock is released.
`404 Not Found`	Unknown `sku`.	
`409 Conflict`	Not enough stock.	Nothing is charged.
`502 Bad Gateway`	The provider rejected our request (400/401 from PayFast: a bug or misconfiguration on our side).	Stock is released.
`503 Service Unavailable`	The provider is down or timed out.	Header `Retry-After: 30`. Stock is released.

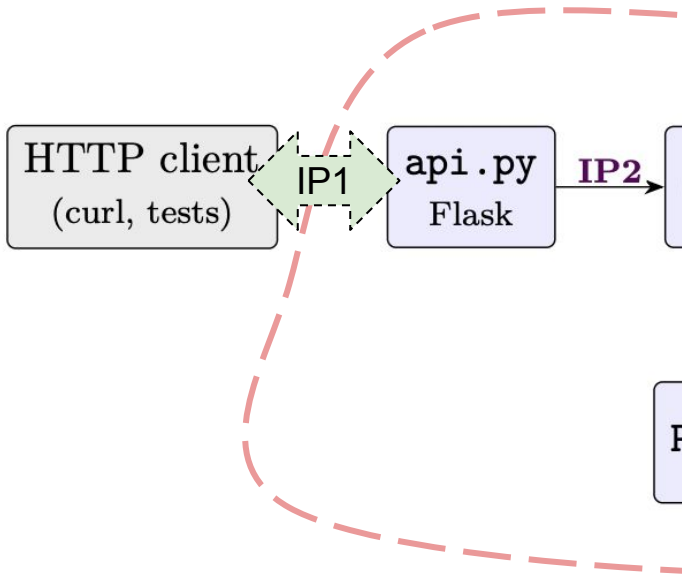
The order resource

```
{"id": 1, "sku": "MUG", "quantity": 2, "total_cents": 2500,
 "customer_email": "ana@example.com", "status": "paid", "charge_id": "ch_1"}
```

`status` is one of `pending`, `paid`, `payment_failed`.

`GET /orders/<id>`

`200 OK` with the order resource, or `404 Not Found`.



The shop. Grey boxes are outside the shop's two components must agree.

R8	Two credentials. A wrong *provider* key is our fault: 502, not "retry later".	IP1,6
R9	Every service exception needs a documented status; Flask's default errors are HTML.	IP1,2
R11	Where is input validated? Can bad input reach the service?	IP1,2

Shop Orders API, v1

The shop's own HTTP API, implemented in `shop/api.py`. Every request needs the header `X-API-Key: <key>`. Every error response is JSON of the form `{"error": "<message>"}`, never an HTML page.

`POST /orders`

Place an order: reserve stock, charge the customer through PayFast (`payment_provider_api.md`), and record the order.

Body (`Content-Type: application/json`)

Field	Type	Rules
`sku`	string	An existing product.
`quantity`	integer	`>= 1`
`customer_email`	string	Must contain `@` .

Responses

Status	When	Extra
`201 Created`	The order was placed and paid.	Body: the order. Header `Location: /orders/<id>` .
`400 Bad Request`	Body is not a JSON object, a field is missing, or a field breaks a rule.	
`401 Unauthorized`	`X-API-Key` missing or wrong.	
`402 Payment Required`	The provider declined the card.	Stock is released.
`404 Not Found`	Unknown `sku` .	
`409 Conflict`	Not enough stock.	Nothing is charged.
`502 Bad Gateway`	The provider rejected our request (400/401 from PayFast: a bug or misconfiguration on our side).	Stock is released.
`503 Service Unavailable`	The provider is down or timed out.	Header `Retry-After: 30` . Stock is released.

The order resource

```
{"id": 1, "sku": "MUG", "quantity": 2, "total_cents": 2500,
 "customer_email": "ana@example.com", "status": "paid", "charge_id": "ch_1"}
```

`status` is one of `pending` , `paid` , `payment_failed` .

`GET /orders/<id>`

`200 OK` with the order resource, or `404 Not Found` .

a HTTP client → api.py: what crosses the wire

LIVE: top-down
`pytest tests/test_api_topdown.py`

PRODUCTION `shop/api.py`

```
def create_app(service, api_key):
    app = Flask(__name__)

    def error(status, message, headers=None):
        return jsonify(error=message), status, headers or {}

1

    @app.before_request
    def require_api_key():
        if request.headers.get("X-API-Key") != api_key:
            return error(401, "missing or invalid API key")
    ...
2       return jsonify(order), 201, {"Location":
                                     f"/orders/{order['id']}"}
    ...
3   @app.errorhandler(404)
    @app.errorhandler(405)
    def json_errors(exc):  # Flask's defaults are HTML pages
        return error(exc.code, exc.name.lower())
```

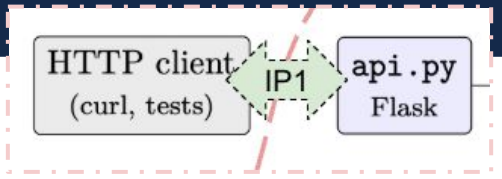
TEST `tests/test_api_topdown.py` R8 R9

```
1 def test_missing_api_key_never_reaches_the_service():
      stub = StubOrderService()
      response = api(stub).post("/orders", json=GOOD_ORDER)
      assert response.status_code == 401
      assert stub.calls == []

2
  def test_success_is_201_with_a_location_header():
      response = api(StubOrderService()).post(
          "/orders", json=GOOD_ORDER, headers=HEADERS)
      assert response.status_code == 201
      assert response.headers["Location"] == "/orders/7"

3 def test_unknown_route_answers_in_json_not_html():
      response = api(StubOrderService()).get(
          "/order/1", headers=HEADERS)    # typo in the path
      assert response.status_code == 404
      assert response.is_json
```

A client never sees a Python exception, only status, headers and body. These tests use Flask's test client against the real routes, so they check the contract in specs/shop_api.md: 401 before anything else runs, 201 + Location, JSON even for errors.



a HTTP client → api.py: validate at the boundary

LIVE: parametrized
`pytest tests/test_api_topdown.py -k invalid`

PRODUCTION `shop/api.py`

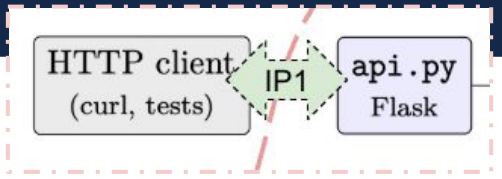
```
@app.post("/orders")
def place_order():
    body = request.get_json(silent=True)
    problem = validate_order(body)
    if problem:
        return error(400, problem)
    ...

def validate_order(body):
    if not isinstance(body, dict):
        return "request body must be a JSON object"
    missing = [f for f in ("sku", "quantity","customer_email")
               if f not in body]
    if missing:
        return f"missing field(s): {', '.join(missing)}"
    if type(body["quantity"]) is not int or body["quantity"] < 1:
        return "quantity must be a positive integer"
    if not isinstance(body["customer_email"], str) \
            or "@" not in body["customer_email"]:
        return "customer_email must be an email address"
```

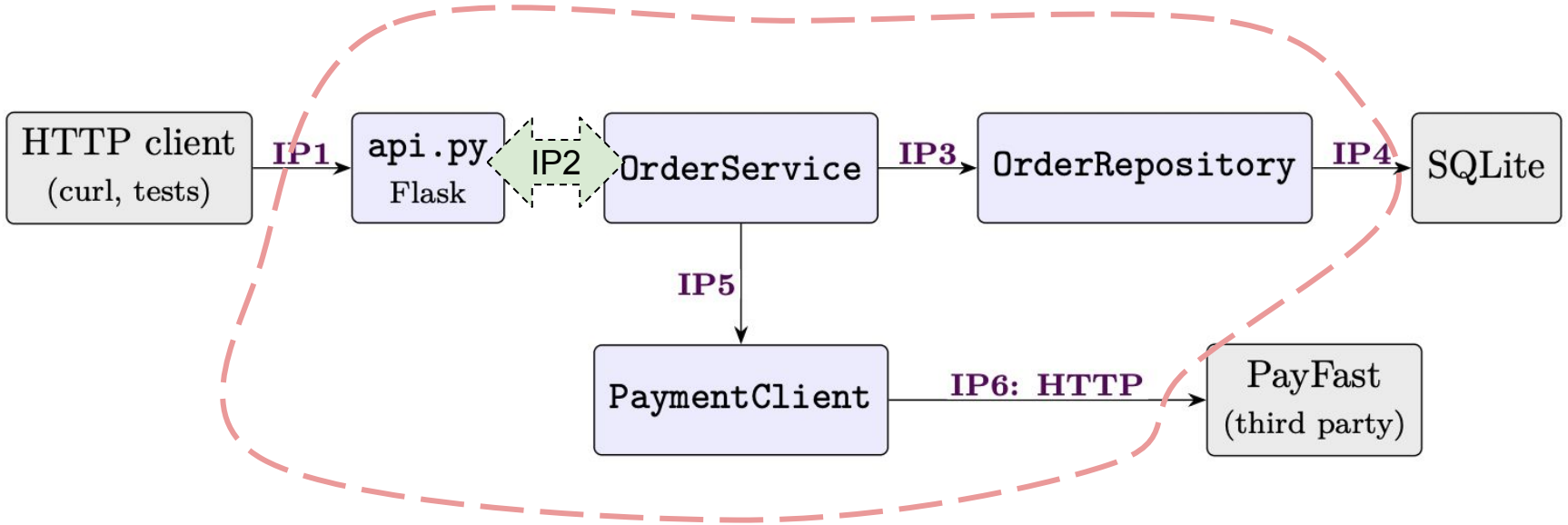
TEST `tests/test_api_topdown.py` **R11**

```
@pytest.mark.parametrize("body, message", [
    pytest.param(["MUG", 2],
        "must be a JSON object", id="not-an-object"),
    pytest.param({"sku": "MUG", "quantity": 2},
        "missing field(s): customer_email", id="missing"),
    pytest.param({**GOOD_ORDER, "quantity": 0},
        "positive integer", id="zero"),
    pytest.param({**GOOD_ORDER, "quantity": "2"},
        "positive integer", id="string-quantity"),
    pytest.param({**GOOD_ORDER, "customer_email": "ana"},
        "email address", id="bad-email"),
])
def test_invalid_bodies_never_reach_the_service(body, message):
    """R11: validate at the boundary."""
    stub = StubOrderService()
    response = api(stub).post("/orders", json=body,
                              headers=HEADERS)
    assert response.status_code == 400
    assert message in response.get_json()["error"]
    assert stub.calls == []
```

One parametrized case per rule. The "string-quantity" case matters most: JSON can carry "2" (a string), and only a check at the boundary stops it from reaching the service as a str. stub.calls == [] proves nothing below was called.



Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

b api.py → OrderService: exceptions become statuses

LIVE: stub raises
`pytest tests/test_api_topdown.py -k errors`

PRODUCTION `shop/api.py` (errors from `shop/errors.py`)

```
    try:
1       order = service.place_order(
            body["sku"], body["quantity"], body["customer_email"]
        )
2   except UnknownProduct:
        return error(404, f"unknown sku: {body['sku']}")
3   except OutOfStock:
        return error(409, "insufficient stock")
4   except PaymentDeclined:
        return error(402, "payment declined")
5   except PaymentUnavailable:
        return error(503, "payment provider unavailable",
                     {"Retry-After": "30"})
6   except PaymentError:      # base class: must come last
        return error(502, "payment provider rejected the request")
```

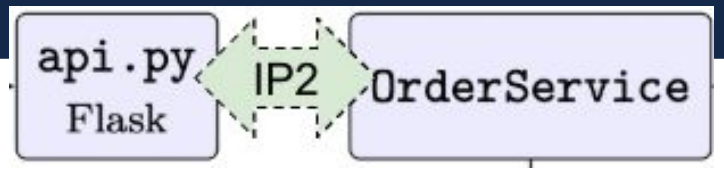
```
# shop/errors.py
class PaymentDeclined(PaymentError): ...
class PaymentUnavailable(PaymentError): ...
```

TEST `tests/test_api_topdown.py` **R9**

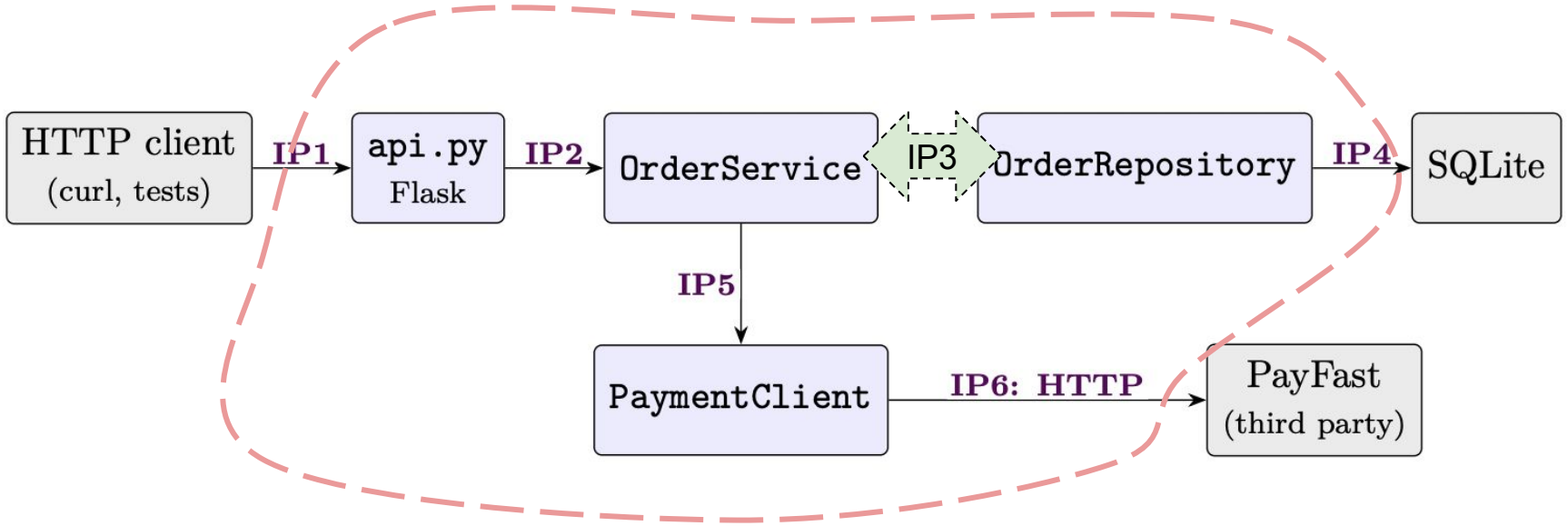
```
1 class StubOrderService:        # no database, no PayFast
      def place_order(self, sku, quantity, customer_email):
          self.calls.append((sku, quantity, customer_email))
          if self.raises:
              raise self.raises
          return {"id": 7, "sku": sku, ...}


  @pytest.mark.parametrize("raised, status", [
2   pytest.param(UnknownProduct("MUG"), 404, id="unknown-sku"),
3   pytest.param(OutOfStock("MUG"), 409, id="out-of-stock"),
4   pytest.param(PaymentDeclined("card_declined"), 402, ...),
5   pytest.param(PaymentUnavailable("...503"), 503, ...),
6   pytest.param(PaymentError("...: 401"), 502, id="our-bug"),
  ])
  def test_service_errors_map_to_the_documented_status(raised,
                                                       status):
      response = api(StubOrderService(raises=raised)).post(
          "/orders", json=GOOD_ORDER, headers=HEADERS)
      assert response.status_code == status
      assert response.is_json and "error" in response.get_json()
```

The exceptions in shop/errors.py are the interface between the two components. Because Declined and Unavailable subclass PaymentError, the order of the except clauses is part of the contract: move PaymentError to the top and two cases fail.



Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

C OrderService → OrderRepository: one transaction

– where it starts and ends

LIVE: real SQLite
`pytest tests/test_order_service.py`

PRODUCTION `shop/service.py`

```
def place_order(self, sku, quantity, customer_email):
    product = self.repository.get_product(sku)
    if product is None:
        raise UnknownProduct(sku)
    total_cents = product["price_cents"] * quantity

    # One transaction: the stock and the pending order are
    # saved together, or (if anything raises) not at all.
2   with self.connection:
1       if not self.repository.reserve_stock(sku, quantity):
            raise OutOfStock(sku)
2       order_id = self.repository.create_order(
            sku, quantity, total_cents, customer_email
        )

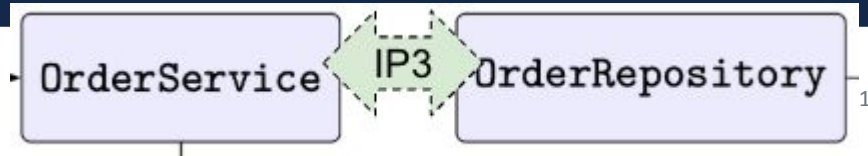
    # ... then charge (point e)
```

TEST `tests/test_order_service.py` **R3**

```
1  def test_out_of_stock_charges_nothing(service, provider,
                                         connection):
       with pytest.raises(OutOfStock):
           service.place_order("TEE", 1, "ana@example.com")
       assert provider.requests == []
       count = "SELECT COUNT(*) FROM orders"
       assert connection.execute(count).fetchone()[0] == 0

2  def test_order_the_database_rejects_leaves_stock_untouched(
       service, provider, stock):
       """R3: the database refuses the order (NOT NULL
       customer_email), so the reservation must be undone too."""
       with pytest.raises(sqlite3.IntegrityError, match="NOT NULL"):
           service.place_order("MUG", 2, customer_email=None)
       assert stock("MUG") == 5      # via a second connection
       assert provider.requests == []
```

The repository never commits; the service decides where a transaction starts and ends. To check that boundary you need a database that can refuse a write (NOT NULL) and a second connection that sees only committed rows: the stock fixture.



c Why a mock can't see a transaction bug

LIVE: broken variant
`SHOP_SERVICE=buggy pytest …`

PRODUCTION **service.py vs. service_buggy.py**

```
# shop/service.py: one transaction
with self.connection:
    if not self.repository.reserve_stock(sku, quantity):
        raise OutOfStock(sku)
    order_id = self.repository.create_order(
        sku, quantity, total_cents, customer_email)
```

```
# shop/service_buggy.py: two transactions
with self.connection:  # KNOWN BUG: commits the stock...
    if not self.repository.reserve_stock(sku, quantity):
        raise OutOfStock(sku)
with self.connection:  # ...before the order insert can fail
    order_id = self.repository.create_order(
        sku, quantity, total_cents, customer_email)

# With customer_email=None: the insert fails, the first
# block is already committed, 2 mugs vanish from stock.
```

SHOP_SERVICE=buggy: what each file reports (R3)

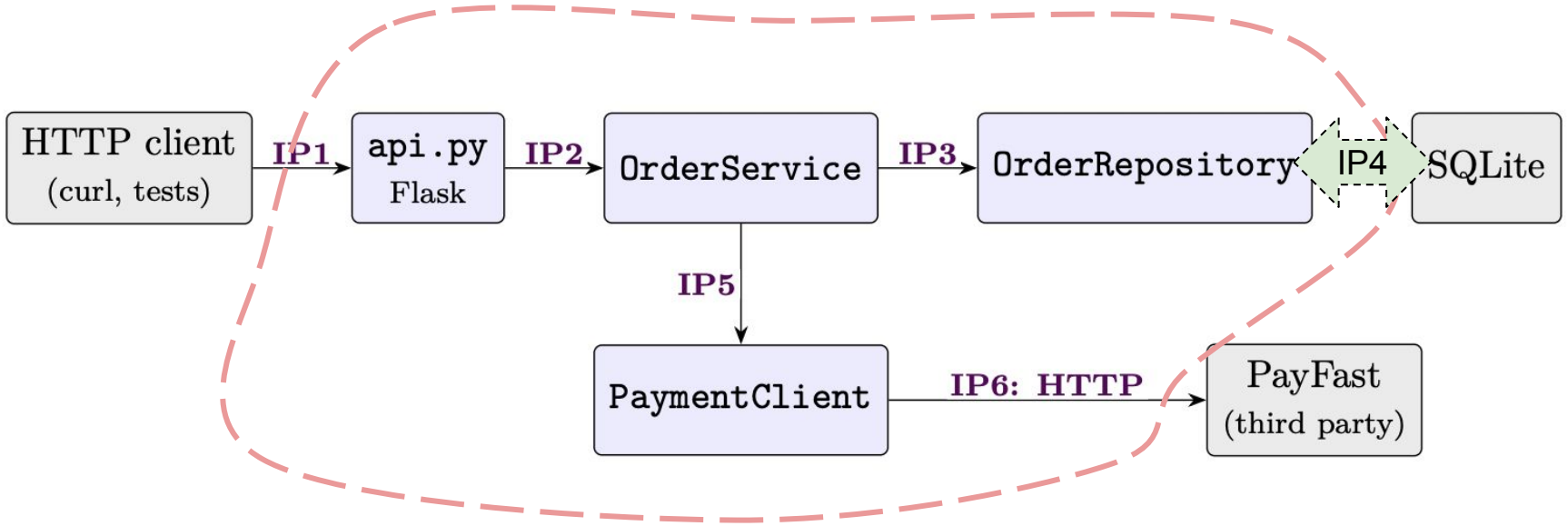
`test_order_service_unit.py` 3 passed — MISSED
Repository is an autospec mock and the connection a MagicMock: no transactions, no NOT NULL. Every call is still made in the right order.

`test_order_service.py` 1 failed: assert 3 == 5 — CAUGHT
Real SQLite rejects customer_email=None; a second connection sees the committed stock: 3, not 5.

`test_api.py` 5 passed — HIDDEN
No API test can trigger it: a missing email is rejected with 400 at point a, before the service runs.

The defect lives between the service and the database, so only a test with a real database at point c can see it. The unit test is still worth keeping: it pins the service's own decisions, fast.

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

d OrderRepository → SQLite: the SQL itself

LIVE: bottom-up driver
`pytest tests/test_repository.py`

PRODUCTION `shop/repository.py`

```
1  def reserve_stock(self, sku, quantity):
       cursor = self.connection.execute(
           "UPDATE products SET stock = stock - ? "
           "WHERE sku = ? AND stock >= ?",
           (quantity, sku, quantity),
       )
       return cursor.rowcount == 1

2  def create_order(self, sku, quantity, total_cents,
                    customer_email):
       cursor = self.connection.execute(
           "INSERT INTO orders (sku, quantity, total_cents, "
           "customer_email) VALUES (?, ?, ?, ?)",
           (sku, quantity, total_cents, customer_email),
       )
       return cursor.lastrowid

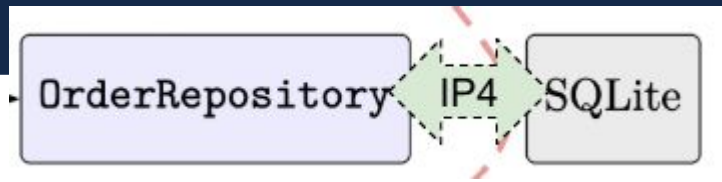
3  def get_order(self, order_id):
       row = self.connection.execute(
           "SELECT * FROM orders WHERE id = ?", (order_id,)
```

TEST `tests/test_repository.py`

```
def test_order_create_read_update_delete(connection,
                                         repository):
2      with connection:
           order_id = repository.create_order(
               "MUG", 2, 2500, "ana@example.com")
3      assert repository.get_order(order_id) == {
           "id": order_id, "sku": "MUG", "quantity": 2,
           "total_cents": 2500, "customer_email": "ana@example.com",
           "status": "pending", "charge_id": None,
       }
       ...  # update -> "paid", "ch_9"; delete -> None

1  def test_reserve_stock_refuses_to_oversell(connection,
                                              repository, stock):
       with connection:
           assert repository.reserve_stock("MUG", 5)
           assert not repository.reserve_stock("MUG", 1)
       assert stock("MUG") == 0
```

Bottom-up: the test is the driver, calling the repository the way OrderService will. Only a real database checks the SQL, the column defaults (status "pending", charge_id NULL) and the stock >= ? guard that stops overselling.



d OrderRepository → SQLite: constraints and commits

LIVE: remove a line
`sed -i.bak '/PRAGMA/d' shop/db.py`

PRODUCTION `shop/db.py`

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS products (
    sku          TEXT PRIMARY KEY,
    ...
    stock        INTEGER NOT NULL CHECK (stock >= 0)
);
CREATE TABLE IF NOT EXISTS orders (
    id           INTEGER PRIMARY KEY,
    sku          TEXT NOT NULL REFERENCES products (sku),
    ...
);
"""

def connect(path):
    connection = sqlite3.connect(path)     # a file, not :memory:
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")   # off by
                                                     # default
    return connection
```

TEST `tests/test_repository.py` R10 R3

```
def test_database_rejects_negative_stock(connection):
    with pytest.raises(sqlite3.IntegrityError,
                       match="CHECK constraint failed"):
        with connection:
            connection.execute(
                "UPDATE products SET stock = -1 WHERE sku = 'MUG'")
```

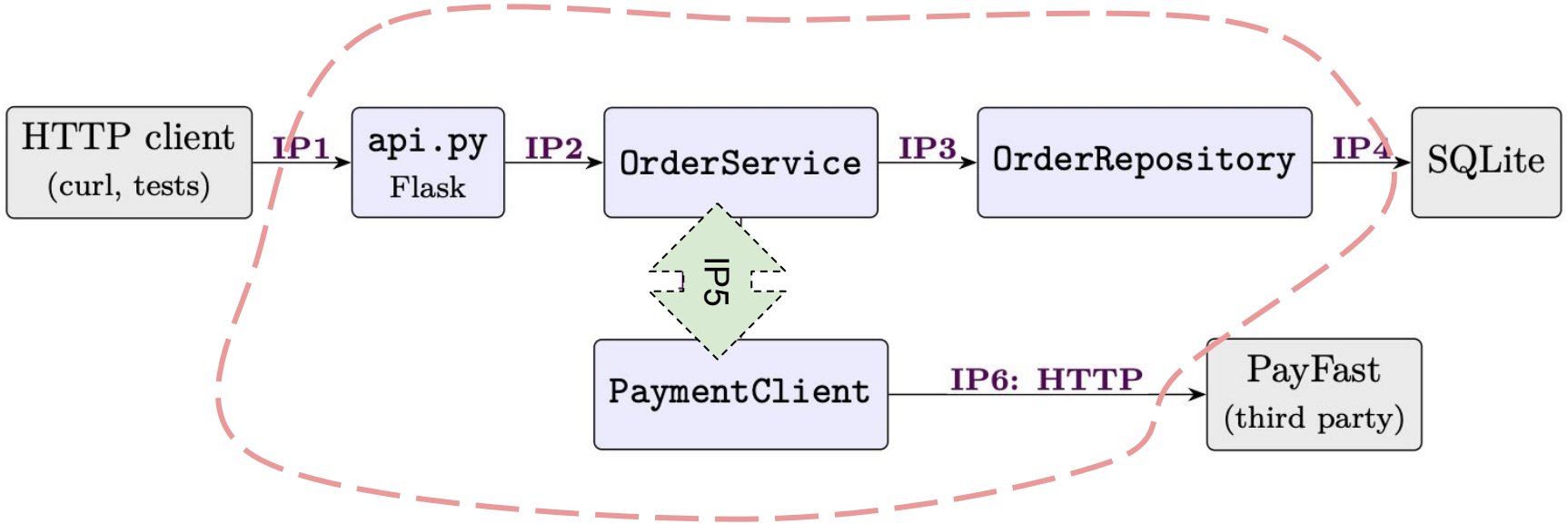
```
def test_database_rejects_an_order_for_an_unknown_product(
        connection, repository):
    """R10: fails if the PRAGMA line is removed."""
    with pytest.raises(sqlite3.IntegrityError,
                       match="FOREIGN KEY constraint failed"):
        with connection:
            repository.create_order("NOPE", 1, 100, "a@b.com")
```

```
def test_uncommitted_changes_are_invisible_to_other_connections(
        connection, repository, stock):
    repository.reserve_stock("MUG", 2)   # not committed yet
    assert stock("MUG") == 5             # a 2nd connection: 5
    connection.commit()
```

Constraints are rules the database enforces whoever calls it. Delete the PRAGMA line and the foreign-key test fails (1 failed, 7 passed). A file in tmp_path, rather than :memory:, is what lets a second connection check what was committed.



Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

e OrderService → PaymentClient: charge, or undo

LIVE: fake PayFast
`pytest tests/test_order_service.py`

PRODUCTION `shop/service.py`

```
1   total_cents = product["price_cents"] * quantity
    with self.connection:
        ...  # reserve stock + create order (point c)

    # The charge cannot join a database transaction: it
    # happens in someone else's system. Undo our side by hand.
1   try:
        charge_id = self.payments.charge(
            total_cents, reference=f"order-{order_id}")
3   except PaymentError:
        with self.connection:
            self.repository.release_stock(sku, quantity)
            self.repository.update_order(order_id,
                                         "payment_failed")
        raise

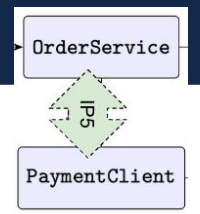
2   with self.connection:
        self.repository.update_order(order_id, "paid", charge_id)
    return self.repository.get_order(order_id)
```

TEST `tests/test_order_service.py` R2 R4

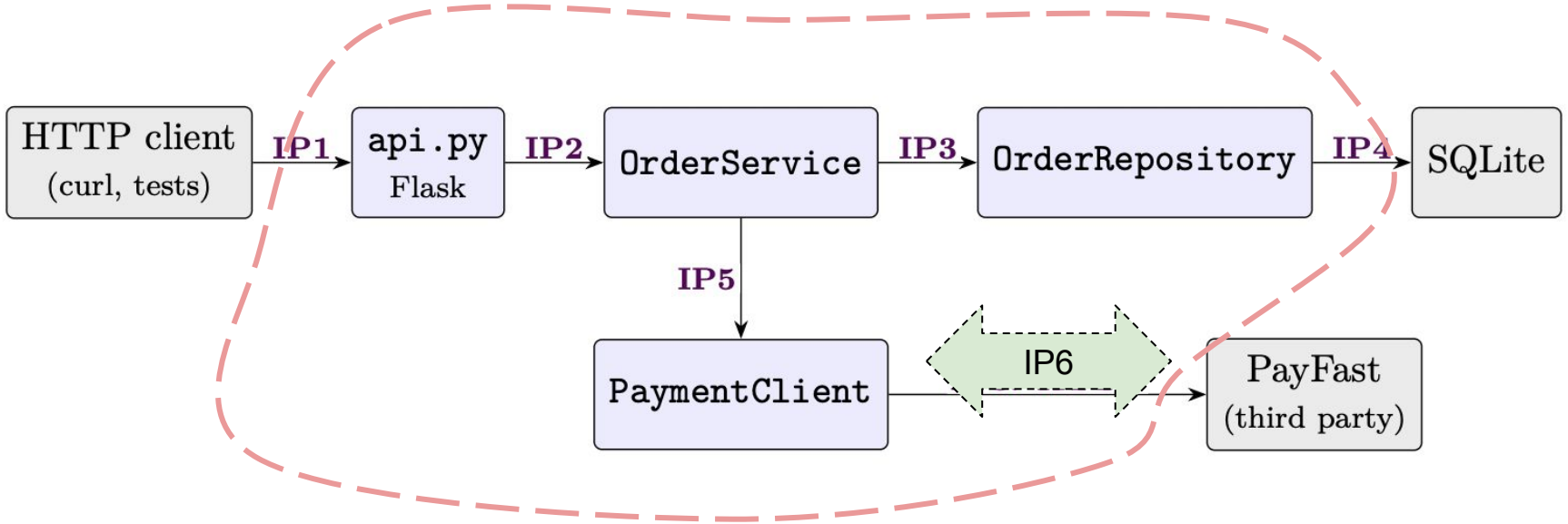
```
    def test_successful_order_charges_exactly_the_order_total(
            service, provider, stock):
        """R2: the amount that flows to PayFast is the amount
        we recorded."""
2       order = service.place_order("MUG", 2, "ana@example.com")
        assert order["status"] == "paid"
        assert order["charge_id"] == "ch_1"
1       assert order["total_cents"] == 2500
2       assert provider.charges[0]["amount"] == order["total_cents"]
        assert stock("MUG") == 3

3   def test_declined_payment_gives_the_stock_back(service,
                                                   provider, stock):
        provider.config["MODE"] = "decline"
        with pytest.raises(PaymentDeclined):
            service.place_order("MUG", 2, "ana@example.com")
        assert stock("MUG") == 5
        assert service.get_order(1)["status"] == "payment_failed"
```

Check both sides of the seam: what the database committed and what the provider received. The interesting failures are disagreements between the two, e.g. 2 mugs gone from stock but no charge, or a charge for an amount we never recorded.



Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

f PaymentClient → PayFast: the request on the wire

LIVE: HTTP-layer stub
`pytest tests/test_payments_stubbed.py`

PRODUCTION `shop/payments.py`

```
    def charge(self, amount_cents, reference):
2       payload = {"amount": amount_cents, "currency": "USD",
                   "reference": reference}
3       headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Idempotency-Key": reference,
        }
        ...
            response = self.session.post(
                f"{self.base_url}/v1/charges",
                json=payload, headers=headers, timeout=self.timeout,
            )
        ...
1       if response.status_code == 201:
            return response.json()["id"]
```

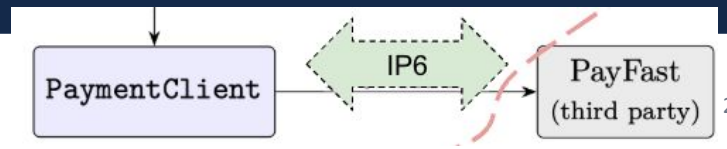
```
2   # shop/payments_v1.py (outdated): KNOWN BUG
        "amount": f"{amount_cents / 100:.2f}",    # "12.50"
```

TEST `tests/test_payments_stubbed.py` — R1 R6 R8

```
    @responses.activate
1   def test_loose_stub_only_checks_the_return_value(client):
        responses.post(CHARGES, json={"id": "ch_1", ...},
                       status=201)
        assert client.charge(1250, reference="order-1") == "ch_1"

    @responses.activate
2   def test_request_matches_the_provider_contract(client):
        responses.post(CHARGES, json={"id": "ch_1", ...},
                       status=201)
        client.charge(1250, reference="order-1")
        sent = responses.calls[0].request
        assert json.loads(sent.body) == {
            "amount": 1250, "currency": "USD",
            "reference": "order-1",
        }
3       assert sent.headers["Authorization"] == "Bearer sk_test_123"
        assert sent.headers["Idempotency-Key"] == "order-1"
```

A stub only knows what its author wrote into it. With SHOP_PAYMENTS=v1 the loose stub (1) still passes; the contract check (2) fails: {'amount': '12.50'} != {'amount': 1250}. Assert on what was sent, not only on what came back.



f PaymentClient → PayFast: a fake that checks

LIVE: real HTTP
`pytest tests/test_payments_live.py`

FAKE **fakes/payment_provider.py**

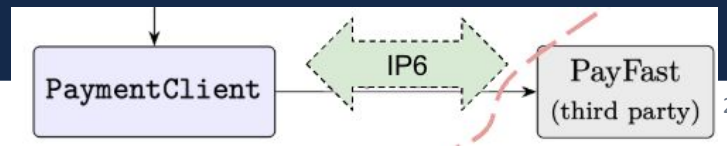
```
@provider.post("/v1/charges")
def create_charge():
    provider.requests.append(request.get_json(silent=True))
    ...
    auth = request.headers.get("Authorization")
    if auth != f"Bearer {secret}":
1       return jsonify(error="invalid_api_key"), 401
    body = request.get_json(silent=True) or {}
    amount = body.get("amount")
    if type(amount) is not int or amount <= 0:
        return jsonify(error="amount must be a positive "
                             "integer number of cents"), 400
    ...
2   key = request.headers.get("Idempotency-Key")
    if key in by_key:
        return jsonify(by_key[key]), 201
    ...
1   charge = {"id": f"ch_{next(ids)}", "amount": amount, ...}
    provider.charges.append(charge)
2   if key:
        by_key[key] = charge
```

TEST **tests/test_payments_live.py** R1 R6

```
1 def test_charge_succeeds_in_integer_cents(payments,
                                             provider):
      """R1: the provider receives 1250 (cents), not "12.50"."""
      assert payments.charge(1250, reference="order-1") == "ch_1"
      assert provider.charges == [
          {"id": "ch_1", "amount": 1250,
           "currency": "USD", "status": "succeeded"}
      ]
```

```
2 def test_same_reference_is_charged_only_once(payments,
                                                provider):
      """R6: what makes the client's retries safe."""
      first = payments.charge(1250, reference="order-1")
      second = payments.charge(1250, reference="order-1")
      assert first == second
      assert len(provider.charges) == 1
```

A fake is a small, working PayFast on a local port. Unlike a stub, it rejects a wrong request the way the real provider would. With SHOP_PAYMENTS=v1: 3 failed, each with 400 amount must be a positive integer number of cents.



f PaymentClient → PayFast: theirs or ours?

LIVE: misbehaving fake
`pytest tests/test_payments_live.py`

PRODUCTION `shop/payments.py`

```
1  RETRYABLE = {500, 502, 503, 504}
   ...
   for _ in range(self.retries + 1):
       try:
           response = self.session.post(..., timeout=self.timeout)
2      except (requests.ConnectionError, requests.Timeout) as exc:
           problem = f"provider unreachable ({type(exc).__name__})"
           continue
1      if response.status_code in RETRYABLE:
           problem = f"provider returned {response.status_code}"
           continue
1      break
   else:
       raise PaymentUnavailable(problem)
   ...
   if response.status_code == 402:
       raise PaymentDeclined(response.json()["error"])
3  raise PaymentError(f"provider rejected the request: "
                      f"{response.status_code} ...")
```

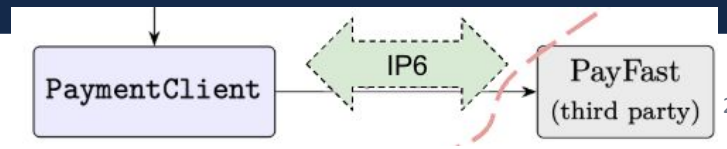
TEST `tests/test_payments_live.py` R5 R7 R8

```
1  def test_outage_is_retried_then_reported_as_unavailable(
           payments, provider):
       provider.config["MODE"] = "unavailable"   # 503, HTML body
       with pytest.raises(PaymentUnavailable, match="503"):
           payments.charge(1250, reference="order-1")
       assert len(provider.requests) == 2      # try + 1 retry

2  def test_slow_provider_times_out(provider):
       provider.config["DELAY"] = 0.5
       impatient = PaymentClient(provider.url, "sk_test_123",
                                 timeout=0.2)
       with pytest.raises(PaymentUnavailable, match="ReadTimeout"):
           impatient.charge(1250, reference="order-1")

3  def test_wrong_api_key_is_our_error_and_is_not_retried(provider):
       bad = PaymentClient(provider.url,"sk_wrong_key", timeout=1.0)
       with pytest.raises(PaymentError, match="401") as caught:
           bad.charge(1250, reference="order-1")
       assert type(caught.value) is PaymentError  # not Unavailable
       assert len(provider.requests) == 1
```

Their failure or ours? A 5xx or a timeout is theirs: retry once, then PaymentUnavailable (503 + Retry-After at the API). A 401 is ours, a misconfigured key: no retry, plain PaymentError (502). The fake makes each case happen on demand.



End to end: crosses a–f, can't say where

LIVE: whole shop
`pytest tests/test_api.py`

TEST **tests/test_api.py**

```
HEADERS = {"X-API-Key": "shop-key"}
ORDER = {"sku": "MUG", "quantity": 2,
         "customer_email": "ana@example.com"}

def test_place_an_order_and_fetch_it_back(client):
    response = client.post("/orders", json=ORDER,
                           headers=HEADERS)
    assert response.status_code == 201
    assert response.headers["Location"] == "/orders/1"
    assert response.get_json() == {
        "id": 1, "sku": "MUG", "quantity": 2,
        "total_cents": 2500, "customer_email": "ana@example.com",
        "status": "paid", "charge_id": "ch_1",
    }
    fetched = client.get(response.headers["Location"],
                         headers=HEADERS)
    assert fetched.status_code == 200
    assert fetched.get_json() == response.get_json()
```

via a, b

via c, d, e, f

via a, b, d

Test file	default	SHOP_PAYMENTS =v1	SHOP_SERVICE=b uggy
test_order_service_unit.py	3 passed	3 passed	3 passed
test_api_topdown.py	14 passed	14 passed	14 passed
test_repository.py	8 passed	8 passed	8 passed
test_payments_stubbed.py	5 passed	**1 failed**	5 passed
test_payments_live.py	6 passed	**3 failed**	6 passed
test_order_service.py	5 passed	**2 failed**	**1 failed**
test_api.py	5 passed	**2 failed**	5 passed

Red = the file sees the defect. Each run uses the correct components except the one the variable swaps in.

test_api.py's v1 failure is assert 502 == 201: something is wrong, somewhere in a–f. The per-point tests above say where (f, the request body). And it misses the buggy service entirely. Keep end-to-end tests few; aim most tests at one point.

Cheat sheet: one seam, one representative test

	Interaction point	Representative test	Defect it catches	Other side is
a	**HTTP client → api.py**	`test_invalid_bodies_never_reach_the_service`	bad input or no API key reaches the service (R11, R8); HTML errors (R9)	Flask test client
b	**api.py → OrderService**	`test_service_errors_map_to_the_documented_status`	wrong status for an exception; except clauses out of order (R9)	stub service
c	**OrderService → OrderRepository**	`test_order_the_database_rejects_leaves_stock_untouched`	stock committed apart from its order (R3)	real SQLite
d	**OrderRepository → SQLite**	`test_database_rejects_an_order_for_an_unknown_product`	foreign keys off (R10); SQL guards; defaults	real SQLite file
e	**OrderService → PaymentClient**	`test_declined_payment_gives_the_stock_back`	wrong amount charged (R2); no undo on failure (R4)	fake PayFast
f	**PaymentClient → PayFast**	`test_request_matches_the_provider_contract`	dollars vs. cents (R1); 5xx, retries, timeouts, keys (R5–R8)	stub, then fake

For your project: draw your system's interaction points first, then name, for each one, what both sides must agree on and the one test that would fail if they stopped agreeing.